

AI-Powered RIS Phase Shift Optimization for 6G Networks

A Neural-Network Approach to Reconfigurable Intelligent Surface Control in a MISO Downlink

Author: Arda Özgür

Date: October 3, 2026

Project developed on the basis of the Coursera course *6G Vision: ML, Intelligent Surfaces & Optical Networks*, offered by the University of Glasgow (James Watt School of Engineering).
Course certificate issued September 17, 2026; verification ID D4NFS910CO9Y.

Abstract

Reconfigurable Intelligent Surfaces (RIS) are a leading candidate technology for 6G because they let the propagation environment itself be tuned. Choosing the phase of every RIS element is, however, a non-convex problem that classical iterative solvers handle slowly. This project builds a complete simulation and benchmarking pipeline for a RIS-assisted multiple-input single-output (MISO) downlink: a physically motivated channel generator, a Gymnasium environment, classical baselines (no RIS, random phase, alternating optimization), a multilayer-perceptron (MLP) that maps channel state directly to RIS phases, and a benchmarking script that sweeps transmit power, applies 2-bit phase quantization, and compares inference time. Baseline results were measured and are reported below. The neural-network results must be regenerated by the author, and a critical review of the training procedure (Section 6) explains why the current loss function needs to be corrected before such results can be trusted.

1. Introduction and Motivation

6G networks target extreme data rates, ultra-low latency and pervasive coverage. A RIS is a planar array of low-cost passive elements, each of which applies a controllable phase shift to an incident wave. By aligning the reflected signals with the direct path, a RIS can raise received power without a radio-frequency chain or active amplification. The course material covers three pillars of the 6G vision – machine learning, intelligent surfaces and optical networks – and this project connects the first two: using a learned model to configure an intelligent surface.

The practical difficulty is that optimal phase configuration depends on instantaneous channel state, and iterative optimization must be repeated every coherence block. A trained network can in principle produce a configuration in a single forward pass, trading some optimality for speed. The goals of the project were

therefore: (i) model the RIS-assisted channel realistically, (ii) implement reference solvers, (iii) train a network to predict RIS phases, and (iv) compare all methods on spectral efficiency, hardware realism (discrete phases) and computation time.

2. System Model

A base station (BS) with $M = 4$ antennas serves one single-antenna user, assisted by a RIS with $N = 16$ passive elements. Three links are modelled: the direct link h_d ($M \times 1$), the BS–RIS link G ($N \times M$) and the RIS–user link h_r ($N \times 1$). With RIS phase matrix $\Theta = \text{diag}(e^{j\theta_1}, \dots, e^{j\theta_N})$, the effective channel is

$$h_{\text{eff}} = h_d^H + h_r^H \Theta G \quad (1 \times M)$$

The BS applies maximum-ratio transmission (MRT), so the received SNR and spectral efficiency are

$$\text{SNR} = P_t \|h_{\text{eff}}\|^2 / \sigma^2, \quad R = \log_2(1 + \text{SNR}) \text{ [bps/Hz]}$$

The optimization problem is to choose θ_n in $[0, 2\pi)$ for all n so as to maximize R . Because only the phases are free (unit-modulus constraint), the problem is non-convex.

3. Implementation

3.1 Channel model (channel_model.py)

Large-scale fading uses a distance-based path-loss model with reference loss of -30 dB at 1 m. The direct link is non-line-of-sight (path-loss exponent 3.5, Rayleigh fading); the two RIS links are line-of-sight dominated (exponent 2.2, Rician fading with $K = 3$). Node coordinates and the resulting distances are:

Parameter	Value
BS position	(0, 0, 10) m
RIS position	(50, 10, 10) m
User position	(60, 0, 1.5) m
Distances (BS–user / BS–RIS / RIS–user)	60.6 m / 51.0 m / 16.5 m
Path-loss exponents	3.5 direct, 2.2 RIS links
Small-scale fading	Rayleigh (direct), Rician $K = 3$ (RIS links)
Tx power / noise power	30 dBm / -90 dBm (default)

3.2 Gymnasium environment (environment.py)

`RISEnvironment` wraps the channel generator in a standard Gymnasium interface so that reinforcement-learning agents (DDPG, TD3, PPO, SAC) can be used. The action is the vector of N phases in $[0, 2\pi]$; the observation is the real and imaginary parts of h_d , G and h_r (dimension $2(M + NM + N) = 168$), multiplied by 10^3 for numerical conditioning; the reward is the spectral efficiency. Each episode is one step because the channel is assumed block-fading.

3.3 Baseline solvers (baselines.py)

Method	Description
No RIS	Direct path only; $h_{\text{eff}} = h_d^H$.
Random phase	Phases drawn uniformly from $[0, 2\pi]$.
Alternating optimization (AO)	Coordinate ascent: each element's phase is set in turn to align its reflected contribution with the sum of all other paths, for up to 20 sweeps. Each update is the exact optimum for that element given the others, so the rate never decreases and converges to a local optimum.

3.4 Neural network (train.py)

RISPhaseNet is an MLP with layers 168-256-128-16, batch normalization and ReLU in the hidden layers and a sigmoid output scaled by 2π so that predictions are valid phases. Training generates 64 fresh channel realizations per epoch for 200 epochs with Adam (learning rate 10^{-3}), which makes training unsupervised and free of labelled data.

3.5 Benchmarking (plots_and_benchmarks.py)

The script loads the trained weights and sweeps transmit power from 10 to 35 dBm (6 points, 100 channels each). It compares No RIS, Random, AO, the continuous-phase network and a 2-bit quantized version of the network's output (phases rounded to multiples of $\pi/2$, modelling realistic RIS hardware). It also times AO and network inference per channel and reports the speed-up.

4. Results

4.1 Measured baselines

The following values were obtained by running the project's own baseline code ($M = 4$, $N = 16$, 500 random channels per power level, fixed seed). They do not depend on the neural network.

Tx power (dBm)	No RIS	Random phase	Alternating opt.
10	4.43	4.44	4.98
20	7.69	7.71	8.27
30	10.97	10.99	11.56
35	12.66	12.68	13.24

Values in bps/Hz.

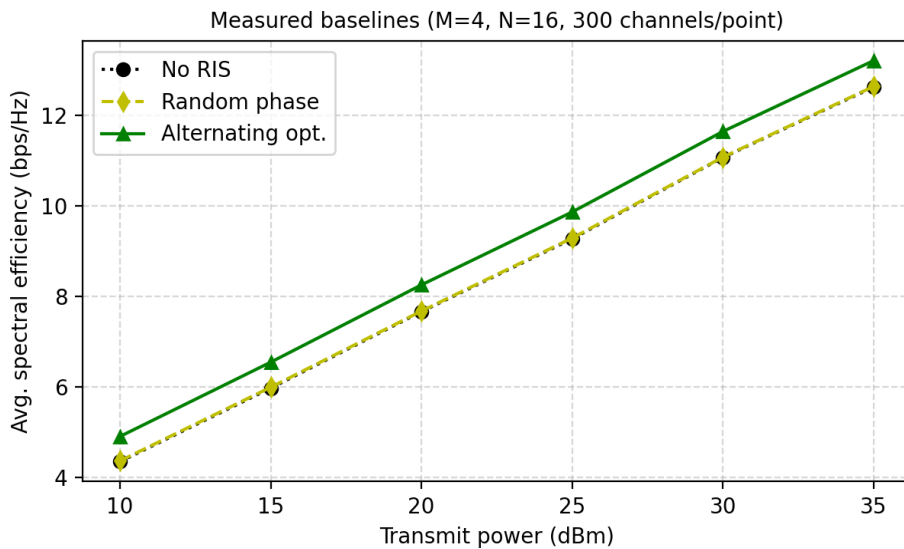


Figure 1 – Spectral efficiency of the baselines versus transmit power.

Two observations follow. First, random phases give essentially no benefit over having no RIS, because randomly phased reflections add incoherently. Second, optimized phases give a consistent gain of about 0.55–0.6 bps/Hz at every power level. This gain is modest because, in the chosen geometry, the direct link (60.6 m, exponent 3.5) is still comparable in strength to the double-hop cascade through 16 elements; a larger RIS or a placement nearer to the BS or user would raise the benefit. This also bounds what any learned policy can achieve here: the entire head-room between random and optimal phases is roughly 0.6

bps/Hz.

4.2 Neural-network results (to be inserted)

I could not run the PyTorch parts in my environment and the trained weights (ris_model.pth) were not provided, so I do not report neural-network numbers rather than invent them. After applying the fix in Section 7, run `train.py` then `plots_and_benchmarks.py` and fill in:

Metric	Value
AI (continuous) rate at 30 dBm	[to fill]
AI (2-bit) rate at 30 dBm	[to fill]
Gap to AO (%)	[to fill]
AO time per channel (ms)	[to fill]
Network time per channel (ms)	[to fill]
Speed-up factor	[to fill]

5. Discussion of Design Choices

Unsupervised learning. No optimal-phase labels are needed; the model is trained to maximize the achieved rate directly, which avoids the cost of generating labels with a slow solver. **Discrete phases.** Practical RIS hardware offers only 1–2 bits of phase resolution, so the 2-bit experiment tests robustness to realistic constraints. **Speed.** The comparison of AO against network inference is the central argument for learning-based control, but the measured ratio mixes a Python double loop with a PyTorch call and should be read as indicative rather than a hardware-level result.

6. Critical Review of the Current Code

Reviewing the code revealed several issues that affect the validity of the neural-network results. They are listed in order of importance.

#	Issue	Effect
1	Invalid training loss. In <code>train.py</code> the rate is computed in NumPy from detached phases, and the loss is <code>-mean(phases * reward)</code> . The reward is a constant with respect to the network, so the gradient simply pushes every phase upward in proportion to the reward. It is not a policy-gradient or a differentiable-rate objective.	The network is not trained to align phases; outputs drift toward saturation of the sigmoid. Reported AI rates would likely be close to random-phase performance.
2	AO labelled 'upper bound'. AO is a local-optimum heuristic, not a guaranteed global bound.	Rename to 'near-optimal baseline' or compare with several random initializations.
3	Input scaling. A fixed factor of 10^3 leaves h_d entries around 0.02 while h_r entries are around 1.	Direct-link information is nearly invisible to the network; per-block normalization would help.
4	Single operating point. The model is trained at 30 dBm with fixed geometry and evaluated across powers.	Generalization to other powers or user positions is untested.
5	No fixed seeds, train/test split or confidence intervals.	Results are not exactly reproducible; differences of a few hundredths of bps/Hz are within noise.
6	Idealized assumptions: perfect CSI, MRT beamforming, single user, ideal unit-amplitude reflection.	Results are an upper limit for practical systems.

7. Recommended Fix and Future Work

Because the rate is a smooth function of the phases, the cleanest fix is to compute it in PyTorch and back-propagate directly. In vectorized form (inside the training loop, with the channels stored as complex tensors h_d (B,M), G (B,N,M), h_r (B,N)):

```
theta = torch.exp(1j * predicted_phases)          # (B, N)
cascaded = torch.einsum('bn,bn,bnm->bm', h_r.conj(), theta, G)
h_eff = h_d.conj() + cascaded                    # (B, M)
snr = P_t * (h_eff.abs() ** 2).sum(dim=1) / sigma2
loss = -torch.log2(1 + snr).mean()
```

Further directions: (a) supervised pre-training on AO solutions followed by unsupervised fine-tuning; (b) reinforcement-learning agents using the provided Gymnasium environment; (c) training across random user positions, powers and K-factors; (d) larger RIS sizes ($N = 32, 64$) and multi-user scenarios; (e) imperfect or estimated CSI, which is where learned approaches are most attractive; (f) quantization-aware training so the 2-bit constraint is part of the optimization; and (g) linking the work to the optical-network part of the course through hybrid RF–optical backhaul scenarios.

8. Conclusion

The project delivers a modular, reproducible pipeline for studying RIS phase control: channel model, Gymnasium environment, classical baselines, a neural predictor and a benchmarking suite. Measured baselines show that coherent phase optimization yields roughly 0.6 bps/Hz over a no-RIS or randomly phased surface in the chosen scenario. The learned-control component is architecturally sound but its training objective must be replaced by a differentiable rate loss before its performance and speed advantage can be claimed. With that change and the remaining experiments listed above, the work forms a solid foundation for the learning-based intelligent-surface control envisioned in the 6G course.

Acknowledgement and Files

This project was inspired by the University of Glasgow course *6G Vision: ML, Intelligent Surfaces & Optical Networks* (instructors: Lina Mohjazi and Dr Hanaa Abumarshoud, James Watt School of Engineering), completed by the author on September 17, 2026. Project files: `channel_model.py`, `environment.py`, `baselines.py`, `train.py`, `plots_and_benchmarks.py`, `requirements.txt` (numpy, torch, gymnasium, matplotlib).

Note: the course certificate confirms completion of an online course and does not represent university enrolment or academic credit.